SAC (Soft Actor-Critic)

First, a story

You are an athlete practicing basketball. After hundreds of sessions you find that driving from the left 45-degree angle has the highest success rate, so you start using only that route. The coach frowns: "Your left layup is solid, but you have exactly one move — once opponents figure out the pattern they will camp there. And — what if a better route exists on the right? You will never find it."

Ordinary RL's way: find one "optimal" route and stick to it; the policy becomes deterministic.

Maximum-entropy RL's way: while keeping the score high, preserve as many viable routes as possible. Randomness is worth pursuing in its own right — it brings exploration, robustness, and makes you harder to counter.

That is the divide between maximum-entropy RL and ordinary RL:

$$\boxed{J(\pi) = \sum_t \mathbb{E}_{(s_t,a_t)\sim\rho_\pi}[r(s_t,a_t) \; + \; \alpha\, H(\pi(\cdot|s_t))]}$$

Formalization

The Shannon entropy (a measure of the policy's randomness):

$$H(\pi(\cdot|s)) = -\mathbb{E}_{a\sim\pi(\cdot|s)}[\log \pi(a|s)]$$

- the uniform distribution maximizes entropy; a deterministic policy has entropy 0;
- $\alpha > 0$ is the temperature, trading off "exploration" against "exploitation"; $\alpha \to 0$ degenerates to ordinary RL.

In plain words: the objective changes from "maximize reward only" to "maximize reward plus α times the randomness". The more random the policy, the bigger the bonus; the better the policy, the higher the reward term — both are optimized simultaneously.

The Soft Bellman Equations

From hard to soft

The ordinary Q function satisfies the Bellman equation:

$$Q(s,a) = r(s,a) + \gamma\, \mathbb{E}_{s'}\left[\max_{a'} Q(s',a')\right]$$

In the maximum-entropy framework the policy no longer takes a max over actions; it takes expectations, with an entropy term added. Define the **soft state value**:

$$\boxed{V_{\text{soft}}(s) = \mathbb{E}_{a\sim\pi}[Q_{\text{soft}}(s,a) - \alpha \log \pi(a|s)]}$$

and the corresponding **soft Q function** (the soft Bellman equation):

$$\boxed{Q_{\text{soft}}(s,a) = r(s,a) + \gamma\, \mathbb{E}_{s'\sim p}[V_{\text{soft}}(s')]}$$

Intuition: V_{soft} carries an extra $-\alpha \log \pi$ term. In high-entropy (random) states $-\alpha \log \pi$ is large and V_{soft} is higher — the algorithm actively rewards states that are "uncertain and explorable".

The optimal soft policy

Under the soft Bellman equation the optimal policy has a closed form:

$$\pi^*(a|s) \propto \exp\left(\frac{1}{\alpha} Q^*_{\text{soft}}(s,a)\right)$$

A Boltzmann (energy) distribution: higher-Q actions get more probability; higher temperature α flattens the distribution (more random); $\alpha \to 0$ degenerates to arg max (deterministic greedy).

SAC's three core mechanisms

Mechanism 1: the tanh-squashed Gaussian policy (reparameterized sampling)

In continuous action spaces the policy outputs a Gaussian $\mathcal{N}(\mu_\phi(s), \sigma_\phi^2(s))$, squashed by tanh into a bounded interval.

$$\boxed{a = \tanh(\mu_\phi(s) + \sigma_\phi(s) \odot \varepsilon)\,, \quad \varepsilon \sim \mathcal{N}(0, I)}$$

Why reparameterize: the randomness ε is "factored out" of the network parameters, so gradients propagate directly to ϕ:

$$\nabla_\phi \mathbb{E}_a[f(a)] = \mathbb{E}_\varepsilon[\nabla_\phi f(\tanh(\mu_\phi + \sigma_\phi \odot \varepsilon))]$$

The tanh correction for log-probabilities (change of variables):

$$\log \pi(a|s) = \log \mathcal{N}(u|\mu_\phi, \sigma_\phi^2) - \sum_d \log\bigl(1 - \tanh^2(u_d)\bigr)$$

with $u = \mu_\phi + \sigma_\phi \odot \varepsilon$ the pre-squash Gaussian sample. The correction $-\sum_d \log(1 - \tanh^2(u_d))$ is always nonnegative: dropping it **underestimates** the action probability (log-prob too small, entropy too large), with the error growing as actions approach the boundary ($|a_d| \to 1$).

Mechanism 2: double Q networks (clipped double Q)

Bootstrapping from a single Q network yields **overestimation bias**: the max (or expectation) latches onto overestimated values and the error compounds. SAC maintains two Q networks $Q_{\theta_1}, Q_{\theta_2}$ and takes the minimum in targets:

$$\boxed{y = r + \gamma \left[\min_{i=1,2} Q_{\bar{\theta}_i}(s', \tilde{a}') - \alpha \log \pi(\tilde{a}'|s')\right], \quad \tilde{a}' \sim \pi(\cdot|s')}$$

Why the minimum? The two networks' initializations and training dynamics differ, so their estimation errors are **not fully correlated**; taking min systematically prefers the more pessimistic of the two — rather underestimate than overestimate, breaking the positive feedback of overestimation.

Each Q network's loss:

$$L_Q(\theta_i) = \hat{\mathbb{E}}\Bigl[\bigl(Q_{\theta_i}(s,a) - y\bigr)^2\Bigr]$$

Mechanism 3: soft target-network updates

Plain DQN hard-copies its target network every fixed number of steps, which can jitter training. SAC smooths with an exponential moving average:

$$\boxed{\bar{\theta}_i \leftarrow \tau\, \theta_i + (1 - \tau)\, \bar{\theta}_i, \quad \tau \ll 1 \text{ (e.g., } 0.005)}$$

$\tau = 0.005$ means the target moves only 0.5% per step — target values evolve very gently and stability improves markedly.

Automatic Entropy Tuning

The problem: how to set α?

α sets the exploration intensity: too large → the policy stays too random and never converges; too small → it degenerates to greedy and loses exploration. Hand-tuning is costly and brittle.

SAC **learns** α by dual gradient descent, targeting a policy entropy equal to a preset target $\mathcal{H}_{\text{target}}$:

$$\boxed{L(\alpha) = \mathbb{E}_{\tilde{a}\sim\pi}[-\alpha\,(\log \pi_\phi(\tilde{a}|s) + \mathcal{H}_{\text{target}})]}$$

Intuition:

- if the current entropy $H(\pi) > \mathcal{H}_{\text{target}}$ (too random) → the gradient lowers α, weakening exploration;
- if $H(\pi) < \mathcal{H}_{\text{target}}$ (too deterministic) → the gradient raises α, strengthening exploration.

α is essentially a Lagrange multiplier enforcing the entropy constraint.

Fig 2. Double Q vs single Q on Pendulum-v1

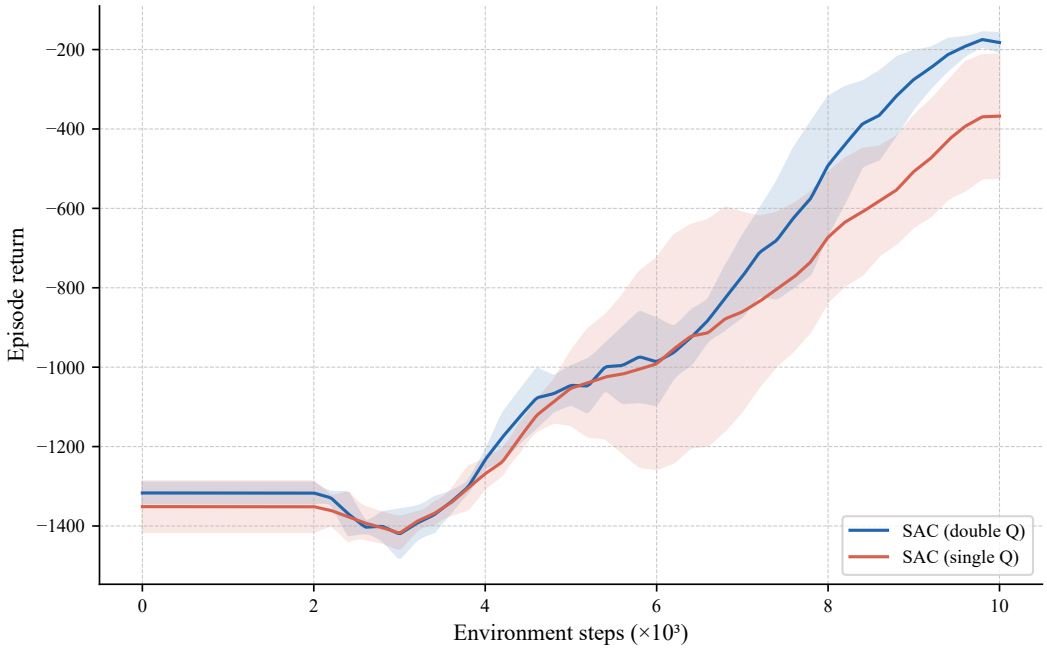

Figure 1: Double vs. single Q ablation (Pendulum-v1, 3 random seeds, shading $\pm 1\sigma$). The double-Q version with the min is higher and more stable.

Fig 3. Soft target update vs hard copy on Pendulum-v1

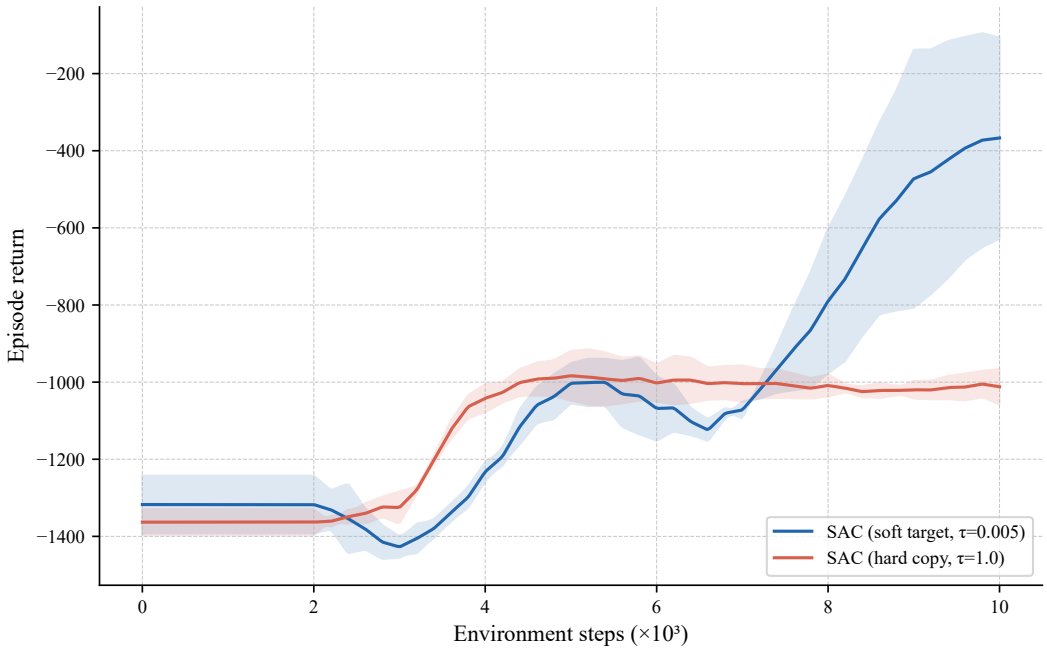

Figure 2: Soft update ($\tau = 0.005$) vs. hard copy ($\tau = 1.0$) ablation (Pendulum-v1, 3 random seeds, shading $\pm 1\sigma$). Soft updates are smoother.

Setting the target entropy

The standard heuristic:

$$\mathcal{H}_{\text{target}} = -\dim(\mathcal{A})$$

For a d-dimensional continuous action space the target entropy is $-d$ (-1 nat per dimension) — equivalent to asking the policy's "effective randomness" per dimension to match a standard normal.

Fig 1. Entropy temperature ablation on Pendulum-v1

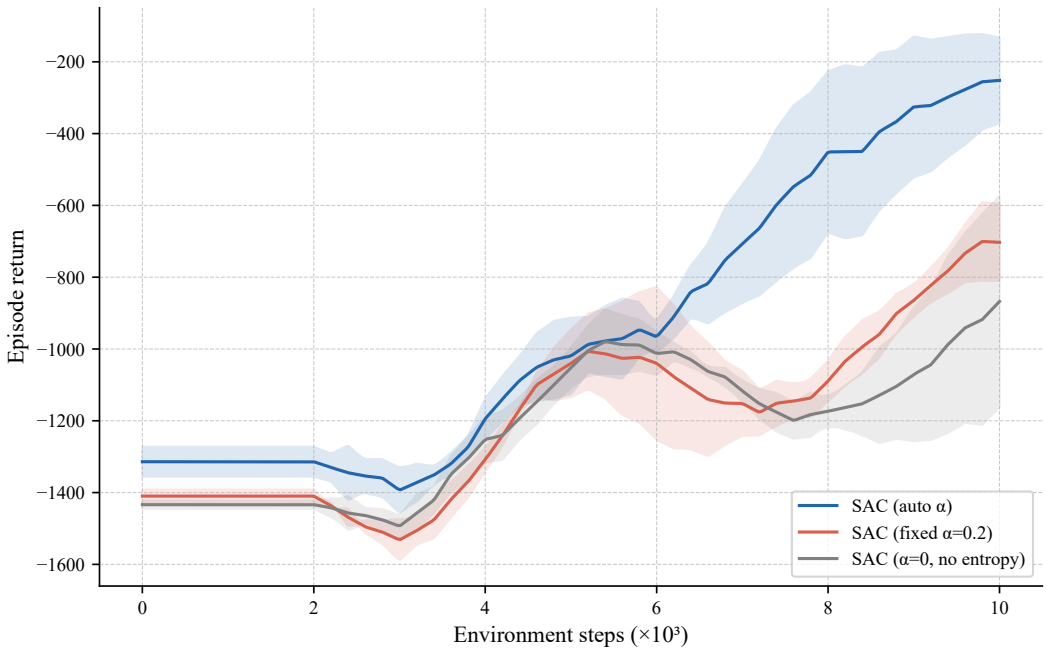

Figure 3: Temperature ablation: automatic α vs. fixed $\alpha = 0.2$ vs. $\alpha = 0$ (Pendulum-v1, 3 random seeds, shading $\pm 1\sigma$). Automatic tuning is usually the most stable.

The full SAC algorithm

Three parallel objectives

$$\boxed{L_Q(\theta_i) = \hat{\mathbb{E}}\Big[\big(Q_{\theta_i}(s,a) - y\big)^2\Big]}$$

$$\boxed{L_\pi(\phi) = \hat{\mathbb{E}}_{s,\tilde{a}}\Big[\alpha \log \pi_\phi(\tilde{a}|s) - \min_i Q_{\theta_i}(s,\tilde{a})\Big]}$$

$$\boxed{L(\alpha) = \hat{\mathbb{E}}_{\tilde{a}}\big[-\alpha\big(\log \pi_\phi(\tilde{a}|s) + \mathcal{H}_{\text{target}}\big)\big]}$$

The actor loss means: maximize "Q value minus α times log-probability" = maximize (expected reward + entropy).

The SAC loop

1. **Initialize**: actor π_ϕ, double critics $Q_{\theta_1}, Q_{\theta_2}$, target networks $Q_{\bar{\theta}_1}, Q_{\bar{\theta}_2}$, replay buffer $\mathcal{B}$, $\log \alpha$;
2. **Collect**: interact with the environment using π_ϕ, storing (s, a, r, s', d) in $\mathcal{B}$;
3. **Every step** (sampling a mini-batch from $\mathcal{B}$):
   (a) compute the soft target y (target networks + next action + entropy correction);
   (b) update the double critics: minimize $L_Q(\theta_1) + L_Q(\theta_2)$;

(c) resample $\tilde{a} \sim \pi_\phi(\cdot|s)$, update the actor: minimize $L_\pi(\phi)$;

(d) update the temperature: minimize $L(\alpha)$;

(e) soft-update the targets: $\bar{\theta}_i \leftarrow \tau\theta_i + (1-\tau)\bar{\theta}_i$.

SAC vs other deep RL methods

SAC vs PPO

Method	Action space	Sample efficiency	Stability	Offline data
PPO	discrete/continuous	medium (on-policy)	high	hard to reuse
SAC	continuous	high (off-policy)	high	reusable, needs conservatism

Why is SAC better suited to continuous actions than PPO?

1. **Off-policy efficiency**: SAC's replay buffer reuses every experience; PPO is on-policy — rollouts are discarded after one use;
2. **High-dimensional continuous actions**: PPO's discretization or Gaussian policies have high variance in high dimensions; SAC's tanh-Gaussian with reparameterized gradients is steadier;
3. **Automatic exploration tuning**: SAC's α adapts; PPO's entropy coefficient c_2 is manual and fixed;
4. **Offline data reuse**: SAC's off-policy structure can reuse historical and offline data; PPO's on-policy structure barely adapts to offline data. Note: reusing offline data is not the same as training offline — vanilla SAC still collapses on purely offline data from extrapolation error and needs CQL/IQL-style conservative modifications (next chapter).

Where this chapter sits in the RL landscape

1. **The TD route** → Q-Learning → DQN (discrete actions, value-based);
2. **The MC route** → REINFORCE → policy gradients (on-policy, high variance);
3. **Actor-Critic** → GAE → PPO (on-policy, clip constraint);
4. **Maximum-entropy RL** → SAC (this chapter: continuous actions, off-policy, automatic entropy tuning).

SAC is the default baseline for continuous control: sample-efficient, stable, self-tuning exploration — the first choice across industry and academia.

References

1. Haarnoja et al. (2018). Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. *ICML*.
2. Haarnoja et al. (2018). Soft Actor-Critic Algorithms and Applications. *arXiv:1812.05905*.
3. Sutton & Barto (2018). *Reinforcement Learning: An Introduction* (2nd ed.). Chapter 13.